



Instrumented & Monitored Cloud Service

Observability Monitoring

Cloud Engineer / Ironhack

Faramarz Karamizadeh, Aug.2026

Agenda

1 1. Architecture & Instrumentation

2 2. Live Demo — Dashboard & Monitoring

3 3. Incident Response Simulation

4 4. Alerting & Response

5 5. Improvements

6 6. Q&A

16 widgets

• Period 60s • SingleValue + Golden Signals + Correlation

5 Custom metrics

request_count • api_latency • error_count •
orders_per_minute • order_value

Architecture

INSTANCE

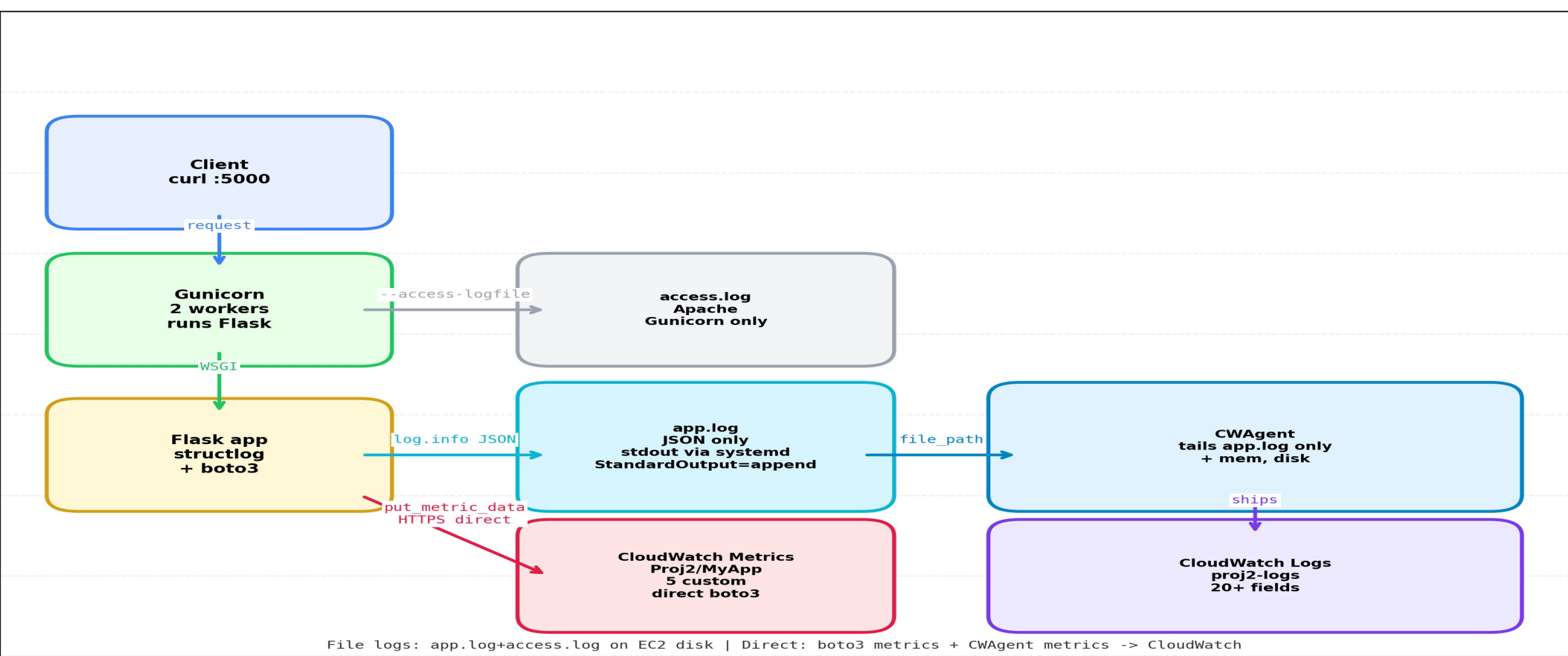
i-01bxxx myapp.service • Flask + Gunicorn 2 sync workers. Systemd service

METRICS

Namespaces: **Proj2/MyApp** (custom) + CWAgent (mem/disk) + AWS/EC2 (CPU/Network) • boto3 put_metric_data • period 60s

LOGGING PIPELINE

systemd StandardOutput → **app.log JSON only**
• gunicorn-error.log • CWAgent → **proj2-logs** log group



Project Highlights

ce-project-2	
docs	
terraform	
.terraform	
modules	
monitoring	
dashboard.json	U
main.tf	M
ec2	
main.tf	
network	
main.tf	
README.md	U
terraform.tfstate	
terraform.tfstate.backup	
.terraform.lock.hcl	
main.tf	
outputs.tf	
variables.tf	
app	
evidence	
tests	
ansible	
roles	
inventory.ini	
playbook.yml	
README.md	U
architecture_diagram.png	U

100% IaC • Terraform + Ansible

Terraform modular + Ansible full stack

VPC, EC2 i-01b04296, SG, IAM-role, CWAgent configs, myapp.service.
Reproducible.

5 Custom Metrics [Business]

orders_per_minute + order_value

request_count api_latency error_count + biz orders_per_minute
order_value via boto3.

6 Warn / Critical Alarms

high-error, high-latency, no-traffic, high-cpu

>5 errors /2x60s, P95 >800ms, no traffic 5m, cpu >85% → SNS proj2-
alerts.

Structured Logging

correlation_id • level • latency_ms

structlog JSON → 20+ fields. X-Corr-ID 51e9ef29 → filter
correlation_id='demo-123'.

Custom Widgets

SingleValue + USE

Rate, Duration, Errors, Saturation unified.

Memory Anomaly alarm and dashboard [USE]

mem_used_percent detection

Infra saturation signal from CWAgent. SingleValue widget + anomaly
band vs latency.

Golden signals

Traffic→ Error→ Latency→ Saturation

EC2 → Gunicorn → Flask → CWAgent → CloudWatch - full traceability

Composite alarms

combining multiple conditions

mem-use-anomaly-detection & high-latency

Correlation Dashboard

Latency vs Traffic vs Errors vs CPU

One view: P95 latency ↔ request_count ↔ error_count ↔ CPU.
Golden + USE.

Custom Metrics — 5 signals via boto3

put_metric_data

request_count

COUNT • Sum • after_request

Every request increments. Fuels Rate & health. Never rely on ALB logs.

api_latency

MILLISECONDS • Avg/Max/P95

latency_ms = (now - g.start)*1000. Duration golden signal.

error_count

COUNT • Sum • if status >=400

Errors golden signal. Triggers high-error alarm when >5 in 2x60s.

orders_per_minute

Business saturation signal. Sum per minute.

BUSINESS

order_value

Amount metric. Tracks revenue proxy, no unit.

None

Alerting — 6 Alarms CloudWatch

error-rate

error_count • Sum >5 • 2 periods of 60s

RED Errors

Stat Sum

Treat missing as not-breaching

latency

api_latency • Average >500ms • 2 periods

RED Duration

Avg

SLO 500ms

memory

request_count • Sum <1 • 300s period • missing data BREACHING

Health-check

1 req /5min min

Missing → ALARM

disk

DISKUtilization • Average >80% • 2 periods 300s • Dim: InstanceId=i-01b042...

USE Saturation

AWS/EC2

Avg 80%

memory anomaly

threshold > 40% • 60s period

Health-check

1 req /5min min

Missing → ALARM

Composite

anomaly + error rate

Health-check

1 req /5min min

Missing → ALARM

- SNS → proj2-alerts + Email notification • IAM proj2-ec2-role needs cloudwatch:PutMetricData only; ListMetrics from Admin • Thresholds tuned for low-traffic demo, not prod scale

Structured Logging

OUTPUT — `app.log` / `journalctl -u myapp`

```
{
  "event": "order_created",
  "order_id": "095cdd18-fd61-48b5-94a8...",
  "amount": 150,
  "correlation_id": "51e9ef29-4528-4582-a712...",
  "timestamp": "2026-08-13T12:58:26.558Z"
}
{
  "event": "request_completed",
  "correlation_id": "51e9ef29...",
  "method": "POST",
  "path": "/orders",
  "status_code": 201,
  "latency_ms": 41,
  "level": "INFO",
  "timestamp": "2026-08-13T12:58:26.599Z"
}
```

Instrumentation — Key Decisions

STRUCTURED LOGGING

structlog JSONRenderer + TimeStamper ISO

```
log.info("request_completed", correlation_id, method, path, status_code, latency_ms, level)
```

Decision: CloudWatch Logs Insights auto-parses JSON → 20+ fields

CORRELATION

before_request → correlation_id

```
g.correlation_id = request.headers.get("X-Correlation-ID", uuid4())
g.start_time = time()
```

Every log line carries same ID → [filter correlation_id="demo-123"](#) finds full flow.

CUSTOM METRICS

boto3 put_metric_data • Namespace Proj2/MyApp

```
cloudwatch.put_metric_data(
    Namespace="Proj2/MyApp",
    MetricData=[{"Name": "request_count", "Value": 1, "Timestamp": utcnow()}])
```

5 signals in after_request + business hook.

ERROR LEVEL

3 levels of error

WARN-ERROR-INFO

RED / USE Analysis — how we triaged

DASHBOARD MAPPING

RED — SERVICE VIEW

Rate	request_count Sum • Traffic timeSeries
Errors	error_count Sum • red spike
Duration	api_latency Avg / Max / P95

USE — RESOURCE VIEW

Utilization	CPUUtilization + mem_used% + disk
Saturation	orders_per_minute • business queue
Errors	same error_count • overlay correlation

Evidence: Injected /fail 20x → Error Rate ↑ 5+ in 60s, Latency flat 30ms, CPU 8% flat → **app error not infra**. Top row = RED single value, Middle = Golden Signals, Bottom = USE, Correlation = all overlayed.

Live Demo Plan — proj2-dashboard

DASHBOARD

proj2-dashboard • 12 widgets • Period 60s •
Region us-east-1 • Auto-refresh 60s

QUICK STATS

SingleValue → timeSeries → correlation. All
three namespaces: Proj2/MyApp + CWAgent +
AWS/EC2

01 — TOP ROW

SingleValue at a glance

Current rate for decision in 3 seconds

request_count Sum

error_count Sum

api_latency Avg

orders/min Sum

02 — MIDDLE

Golden Signals

RED mapping: traffic, errors, duration, saturation

Traffic request_count

Errors error_count

Latency Avg/Max

CPU + Orders

03 — BOTTOM

EC2 USE Resources

Utilization + Saturation. Disk uses SEARCH

CPUUtilization

mem_used_percent

Network In/Out

SEARCH disk% → [[]]

04 — CORRELATION

Latency vs Traffic vs Errors vs CPU

Overlay all in one graph. When /fail injected: Error spikes,
Latency flat, CPU flat → app error not infra

api_latency

request_count

error_count

CPU %

What Would I Do Differently?

01 — INSTRUMENTATION

Use OpenTelemetry SDK, not manual boto3 put_metric_data

Dimensions out of the box, context propagation, no try/except hand-rolled. OTEL → CloudWatch exporter keeps native but adds trace id.

02 — USING GRAFANA

Using a third-parthy opensource dashboard

It could be a good practice for multi-cloud environment with no dependency on Cloudwatch

Thank You

Q&A
